ConfuserNext

User & Project Guide

Covers project file format (.crproj), declarative obfuscation (Obfuscation attribute), pattern expressions, and the full protection and packer reference.

1. Overview

ConfuserNext is an open-source protector (obfuscator) for .NET applications. It is a modernized continuation of ConfuserEx, which was itself the successor of the original Confuser project. ConfuserNext brings that codebase to current .NET tooling: SDK-style projects, and multi-targeting across .NET Framework 4.8, .NET 8.0, and .NET 10.0.

Supported protections at a glance:

- Symbol renaming (including WPF/BAML-aware renaming)
- Protection against debuggers and profilers
- Protection against memory dumping
- Protection against tampering (method body encryption)
- Control flow obfuscation
- Constant and resource encryption
- Reference-hiding proxies
- Disabling decompilers (anti-ILDasm marker)
- Invalid metadata injection
- Compressing/embedding output (packer)

Important runtime note: the compressor packer's multi-module stub technique relies on System.Reflection.Assembly.LoadModule, which .NET Framework supports but CoreCLR (.NET 5 and later, including .NET 8/10) does not implement — it throws NotImplementedException at runtime. The packer still builds successfully against any target, but its output currently only runs on .NET Framework. See section 7.2.

2. Getting Started

Run the CLI against a project file:

```
ConfuserNext.CLI <path to project file>
```

Or point it directly at one or more assemblies with an output directory:

```
ConfuserNext.CLI -n -o=<output directory> <module1.dll> [module2.dll ...]
```

Useful command-line switches:

Switch	Meaning
-n | -noPause	Do not pause for a keypress after finishing.
-o | -out=<dir>	Output directory (direct-module mode only).
-probe=<dir>	Additional directory to search for dependency assemblies.
-plugin=<path>	Load an additional protection/packer plugin assembly.
-debug	Generate debug symbols (.pdb) for the protected output.

3. Project File Format (.crproj)

A ConfuserNext project is an XML document (namespace http://confuser.codeplex.com) describing which modules to protect and which protections/packer to apply. The authoritative schema is ConfuserNext.Core/Project/ConfuserPrj.xsd.

3.1 Element <project>

The root element.

Attribute	Meaning
outputDir	Directory where ConfuserNext stores the output files.
baseDir	Base directory for all relative paths in the document. If this value is itself relative, resolution depends on the host (the CLI resolves it relative to the project file's own location).
seed	Seed for the pseudo-random generator used throughout protection.
debug	Whether debug symbols (.pdb) are generated. Currently unused.

Child element	Meaning
rule	A global protection rule applied to all modules. Optional, repeatable.
packer	The packer setting. Optional.
module	Settings for one module to protect. Repeatable.
probePath	A directory ConfuserNext searches for dependency assemblies. Repeatable.

3.2 Element <module>

Attribute	Meaning
path	Path of the module to protect.
snKey	Path to the Strong Name Key used to sign the module. Optional.
snKeyPass	Password for the SNK, if a PFX container is used. Optional.

Child element: rule — protection rules applied to this specific module.

3.3 Element <rule>

Determines how protections are applied to the components it matches.

Attribute	Meaning
inherit	Whether this rule inherits settings from earlier rules. Default: true.
pattern	The pattern expression selecting which components this rule applies to (see section 5).
preset	The protection preset for this rule: none, minimum, normal, aggressive, or maximum. Default: none.

Child element: protection — repeatable.

3.4 Elements <protection> and <packer>

Attribute	Meaning
id	Identifier of the protection or packer (see sections 6 and 7).
action	"add" or "remove", default "add".

Child element: argument — repeatable.

3.5 Element <argument>

Attribute	Meaning
name	Name of the argument.
value	Value of the argument.

3.6 How rules are applied

Rules are applied from global rules (in the project element) to local rules (in each module element), in document order. ConfuserNext keeps a running set of protection settings for every component and, for each rule in turn:

1. If the component does not match the rule's pattern, skip the rule.

2. If the rule does not inherit (inherit="false"), clear the component's current settings.

3. Apply every protection included in the rule's preset.

4. For each explicit <protection>/<packer> setting in the rule: if action="remove", remove it from the component's settings; if action="add" (default), add/replace it.

3.7 Example

```
<?xml version="1.0" encoding="utf-8"?>
<project baseDir="." outputDir="Confused" xmlns="http://confuser.codeplex.com">
    <rule preset="normal" pattern="true">
        <protection id="anti tamper" action="remove" />
        <protection id="rename">
            <argument name="mode" value="unicode" />
        </protection>
    </rule>
    <packer id="compressor" />
    <module path="MyApp.exe" />
</project>
```

4. Declarative Obfuscation

Besides the project file, protection settings can be attached directly to source code using the standard BCL attribute System.Reflection.ObfuscationAttribute, applied to the assembly, a type, or a member. ConfuserNext reads these attributes at mark time and folds them into the same rule-based settings described in section 3.6.

4.1 Attribute semantics

Combination	Effect
ApplyToMembers = true	The settings on this item also become the base settings for its children.
Exclude = true	No protection is applied to this item.
ApplyToMembers = false, Exclude = false	The protection settings in Feature apply only to this item.
ApplyToMembers = true, Exclude = true	This item and all its children receive no protection.

The Feature property carries a semicolon-separated mini-language built from the same vocabulary as a project rule: an optional preset(...), then one or more +protection-id / -protection-id toggles, or a namespace/type-scoped block.

4.2 Feature syntax examples

```
generate debug symbol:true
random seed:ABCDEFG
strong name key:C:\key.snk
strong name key password:hunter2
packer:compressor(mode=dynamic)
namespace 'ConfuserNext.CLI':preset(normal);+rename;anti
tamper(mode=jit,key=dynamic);-anti debug
preset(none);+rename;
```

4.3 Usage examples

```
[assembly: Obfuscation(Exclude = false, Feature = "preset(minimum);+ctrl flow;-anti
debug;+rename(mode=letters,flatten=false);")]
[assembly: Obfuscation(Exclude = false, Feature = "random seed: Hello!")]
[assembly: Obfuscation(Exclude = false, Feature = "namespace 'Test':-rename")]
namespace Test {
    [Obfuscation(Exclude = false, Feature = "constants")]
    class Program {
        public static void Main() {
            Console.WriteLine("Hi");
        }
    }
}
```

5. Pattern Expressions

The pattern attribute of a <rule> (section 3.3) is a small boolean expression language, evaluated once per candidate component (module, type, or member). If it evaluates to true, the rule applies to that component.

5.1 Literals and operators

Expression	Meaning
true / false	Always matches / never matches.
'...'	A string literal, used as a function argument.
A and B	True only if both A and B are true.
A or B	True if either A or B is true.
not A	True if A is false.

5.2 Functions

Function	Matches
name('X')	The component's simple name equals X.
full-name('NS.Type')	The component's full (namespace-qualified) name equals the argument.
namespace('NS')	The component's namespace equals the argument.
module('Name.dll')	The component's containing module name equals the argument. Note: this is the module's own metadata name, which can differ from the file name on disk (e.g. a console application module is typically named "App.exe" internally even if the file is saved as "App.dll").
decl-type('NS.Type')	For a member, its declaring type's full name equals the argument.
member-type('regex')	A regular expression matched against a composed string describing the component kind (e.g. "type ", "method ", "field ", "property ", "event ", or sub-kinds like "propertym getter ").
is-public()	The component (and all its enclosing types) is publicly visible. Takes no arguments.
is-type('regex')	A regular expression matched against a composed string describing a type's nature (e.g. "enum ", "interface ", "valuetype ", "delegate ", "abstract ", "nested ", "serializable ").
inherits('NS.BaseType')	The type (walking up the inheritance chain) derives from the given fully-qualified base type name.
has-attr('NS.AttrType')	The component carries a custom attribute of the given full type name.
match('regex')	A regular expression matched against the component's full CIL signature string.
match-name('regex')	A regular expression matched against the component's simple name.
match-type-name('regex')	A regular expression matched against the simple name of the component's type (the type itself, or the declaring type for a member).

5.3 Examples

```
true
name('Program')
member-type('type ') and full-name('NS.Type')
namespace('MyApp.Internal') or match-name('.*Legacy.*')
not has-attr('System.ObsoleteAttribute')
```

6. Protections Reference

Each protection is identified by the id used in a <protection id="..."> element or a +id / -id toggle in a Feature string. The Preset column shows the lowest preset level that includes the protection by default; a rule using that preset or higher applies it automatically unless explicitly removed. "None" means the protection is never included by a preset and must be added explicitly.

`rename` — preset: Minimum

Renames types and members to obfuscated identifiers. The only protection that most projects cannot disable without losing all benefit of obfuscation; also drives WPF/BAML resource renaming when applicable.

Parameter	Values	Meaning
mode	letters | ascii | unicode | ...	Character set used for generated names (see RenameMode).
flatten	true | false (default true)	Whether renamed members are flattened into the global namespace where possible.
renameArgs	true | false (default true)	Whether constructor/method argument names are also renamed.
renPdb	true | false (default false)	Whether to rename symbols in the generated PDB as well.
rickroll	true | false (default false)	A long-standing ConfuserEx easter egg; module-level only.
forceRen	true | false (default false)	Forces renaming of an item that would otherwise be skipped (e.g. public API surface).

`anti debug` — preset: Minimum

Makes the protected assembly detect and refuse to run under a debugger or profiler.

Parameter	Values	Meaning
mode	safe | win32 | antinet (default safe)	safe: managed-only Debugger.IsAttached checks plus a COR_ENABLE_PROFILING environment check. win32: adds native IsDebuggerPresent/OutputDebugString checks (Windows only). antinet: adds anti-managed-debugger/profiler COM-interface checks.

`anti ildasm` — preset: Minimum

Marks the assembly so ildasm.exe (and tools honoring the same flag) refuse to disassemble it. A single metadata flag; easily bypassed by other tools, but free to apply.

`anti dump` — preset: Maximum

Detects the assembly being dumped from a live process's memory (a common way to defeat packers) and corrupts the in-memory PE header to make the dump unusable.

`anti tamper` — preset: Maximum

Encrypts method bodies and decrypts them on demand at runtime, detecting modification of the output file.

Parameter	Values	Meaning
mode	normal | jit (default normal)	normal: decrypts each protected method's body once, in bulk, at module load. jit: hooks the JIT compiler and decrypts each method body lazily, right before it is first compiled.
key	normal | dynamic (default normal)	How the decryption key is derived: normal uses a fixed derivation; dynamic generates a randomized derivation routine per build (see ConfuserNext.DynCipher).

`constants` — preset: Normal

Encrypts string, numeric, and array-initializer constants, replacing each literal with a call into a generated decoder.

Parameter	Values	Meaning
mode	normal | dynamic | x86 (default normal)	normal: a fixed decoder routine. dynamic: a randomized decoder generated per build. x86: a native x86 decoder routine (Windows/x86 only).
elements	letters from S, N, P, I (default "SI")	Which constant kinds to encode: S = strings, N = numbers, P = primitives usually left alone (0/1/-1 etc.), I = array initializers.
decoderCount	integer (default 5)	Number of distinct decoder routine variants generated per module, to avoid a single recognizable decoder signature.

`resources` — preset: Normal

Encrypts embedded resources and (when combined with the compressor packer) merged dependency assemblies, decrypting them on first access at runtime.

Parameter	Values	Meaning
mode	normal | dynamic (default normal)	Same meaning as the constants protection's mode parameter.

`ctrl flow` — preset: Normal

Obfuscates control flow by flattening methods into a dispatcher driven by an opaque predicate, making the original logic harder to follow in a decompiler.

Parameter	Values	Meaning
type	switch | jump (default switch)	The flattening technique: a switch-based state machine, or an indirect jump table.
predicate	normal | expression | x86 (default normal)	How the dispatcher's state transitions are obscured: normal uses simple integer arithmetic; expression uses a randomly generated, invertible expression tree (see ConfuserNext.DynCipher); x86 emits native x86 code for the predicate (Windows/x86 only).
intensity	0-100 (default 60)	Percentage of eligible blocks that get flattened.
depth	integer (default 4)	Expression-predicate complexity; only meaningful with predicate=expression.
junk	true | false (default false)	Adds extra unreachable junk blocks to the dispatcher.

`ref proxy` — preset: Normal

Replaces direct method calls, field accesses, and object-construction call sites with indirect calls through generated proxy methods or delegate fields, hiding the real target from a quick decompile.

Parameter	Values	Meaning
mode	mild | strong (default mild)	mild: lightweight per-call-site proxy methods. strong: delegate-field-based proxies with an encoded key feeding the proxy construction, more resistant to static analysis but with higher overhead.
encoding	normal | expression | x86 (default normal)	How the proxy's target/key is encoded: normal is a direct reference; expression uses a generated invertible expression; x86 emits native x86 code (Windows/x86 only).
internal	true | false (default false)	Whether calls to members within the same assembly are also proxied (by default only calls crossing into other assemblies are).
typeErasure	true | false (default false)	Further erases static type information from the proxy signature where possible.
depth	integer (default 3)	Expression-encoding complexity; only meaningful with encoding=expression.

`invalid metadata` — preset: None

Injects metadata that real .NET tooling tolerates but that many third-party decompilers and disassemblers choke on. Not included in any preset because of the compatibility risk; opt in explicitly.

7. Packer

A packer post-processes the whole protected output, after every module has already been protected individually. ConfuserNext ships one packer.

7.1 compressor

Compresses (LZMA) the protected main module and embeds it, along with any dependency modules selected for embedding, inside a small bootstrap stub assembly that decompresses and loads the real payload at startup.

Parameter	Values	Meaning
key	normal | dynamic (default normal)	Same meaning as the equivalent protection parameters: dynamic generates a randomized key-derivation routine per build.
compat	true | false (default false)	Compatibility mode: skips the multi-module embedding technique described below, at the cost of some of the packer's size/obfuscation benefit.

7.2 Known limitation: .NET Framework-only at runtime

This is a real, currently unresolved limitation, not a configuration mistake.

In its default (non-compat) mode, the compressor embeds the protected module as a second module inside a multi-module assembly, and its bootstrap stub loads it via System.Reflection.Assembly.LoadModule at startup. Multi-module assemblies, and LoadModule specifically, are a .NET Framework-only CLR feature: CoreCLR (the runtime behind .NET 5 and every later major version, including .NET 8 and .NET 10) never implemented it, and LoadModule throws System.NotImplementedException unconditionally.

Practically: ConfuserNext itself builds compressor-packed output successfully regardless of which .NET version it is hosted on or which TFM the target application declares, but that output will only actually run if the target application is a .NET Framework application (or is run under .NET Framework compatibility). A .NET 8/10 console or service application packed with the compressor will throw NotImplementedException the moment it starts. compat="true" mode does not use LoadModule and is unaffected by this.

8. Further Reading

- docs/ProjectFormat.md — the condensed Markdown version of section 3, kept in the repository for quick reference alongside the source.
- ConfuserNext.Core/Project/ConfuserPrj.xsd — the authoritative XML schema for .crproj files.
- additional/ — example .crproj files for real-world applications (ILSpy, Paint.NET).